Database Design

How the attendance data is stored, and why it is stored that way

Project	RFID + Fingerprint Attendance and Access Control
Sprint / Story	Sprint 1 — US_2003 (web development side)
Author	Akshit Saini
Date	1 September 2026
Version	0.1 — draft

1. Contents

- 1. Contents
- 2. Why this document exists
- 3. What the project description forced me to store
- 4. The tables
- 5. Column detail
- 6. Decisions I made, and what I rejected
- 7. How to recreate the database in MySQL Workbench
- 8. The ER diagram
- 9. Questions I still have

2. Why this document exists

The hardware team is building the ESP32 reader. My side of the project is the web part: the database, the server that decides who gets in, and later a small dashboard. Before writing any server code I wanted the data model settled, because every later decision depends on it.

I am writing this in my own words as I go, partly so the team can review it and partly so I actually understand what I built. Where I was unsure I have said so instead of pretending.

3. What the project description forced me to store

I went through the complaints in the description one at a time and asked what each one means for the database. That is where the tables came from — I did not start by drawing boxes.

Complaint in the description	What it means for the data
Nothing captures the exit	Every event needs a direction, not just a time. So access_events has a direction column.
Could not account for everyone in a fire drill	Need a live who-is-inside, derived from the events. That is the v_occupancy view.
A record of cards, not of people	The card and the person must be separate. people and credentials are two tables, not one.
Every card opens every door	Need per-person, per-room permission. That is access_rules.
Nobody tracks when access should stop	Credentials need valid_from / valid_until and a revoked flag.

4. The tables

Table	Holds	Why it exists
people	employee code, name, department, type, active flag	The human being. A card is not a person; this is the identity.
credentials	card UID, fingerprint slot, valid_from, valid_until, revoked	What a person presents. Lifecycle lives here, so a lost card is revoked without touching the person.
rooms	code, name, restricted flag	The spaces being accounted for. Restricted means card and finger.
devices	device key, label, room, firmware version, last seen	One ESP32 per door. Tells the server which door is reporting.
access_rules	room plus person or department, fingerprint_required, expiry	Who may go where. This is the answer to 'every card opens every door'.
access_events	person, credential, device, room, direction, decision, reason, both timestamps, device_event_id	The append-only log. Everything else in the system is derived from this.
v_occupancy (view)	last allowed event per person per room where direction is IN	Live occupancy and the fire drill muster list.
users	dashboard email, password hash, role	Logins for the web interface. Nothing to do with the people who tap cards.

5. Column detail

people

Column	Type	Notes
id	INT UNSIGNED, PK	Auto increment
employee_code	VARCHAR(20), unique	EMP001, CON001, VIS001 — the human-readable key
full_name	VARCHAR(120)	
department	VARCHAR(60)	Indexed, because department-wide access rules look it up
person_type	ENUM(employee, contractor, visitor)	Visitors and contractors have expiring credentials
is_active	TINYINT(1)	Someone who has left keeps their history but cannot enter

credentials

Column	Type	Notes
person_id	FK to people, ON DELETE CASCADE	A credential is meaningless without its owner
card_uid	VARCHAR(32), unique, nullable	PN532 UID as uppercase hex. Nullable because someone could be fingerprint-only
fingerprint_slot	SMALLINT, unique, nullable	The R307S template ID. The sensor stores the template, the database stores only the slot number
valid_from / valid_until	DATETIME	NULL valid_until means no expiry. A visitor pass gets today's date
is_revoked / revoked_reason	TINYINT(1) / VARCHAR	A lost card is revoked, never deleted, so old events still make sense

access_events

Column	Type	Notes
direction	ENUM(IN, OUT)	Decided by the server from the previous event, not by the device
decision	ENUM(ALLOWED, DENIED)	Denials are stored too — they are the interesting ones
reason_code	ENUM of nine values	Nine values, from OK through UNKNOWN_CARD to DEVICE_INACTIVE (full list in the API contract)
scanned_at	DATETIME	The device clock
received_at	DATETIME	The server clock. Two columns so clock drift is visible instead of hidden
device_event_id	VARCHAR(48)	Unique together with device_id. This is what makes a retried offline batch safe
person_id	FK, nullable	NULL when the card is unknown — the raw card_uid is still stored

6. Decisions I made, and what I rejected

- No is_inside flag on people. I nearly added one because it is the obvious thing. A flag can drift out of step with the log — one missed event and it lies forever. A view over the events cannot. Slower to query, always correct.
- Events are append-only. The application never issues UPDATE or DELETE on access_events. If a record is wrong, a correcting event is added. An audit log you can edit is not an audit log.
- Two timestamps, not one. If I only kept one, a device with a wrong clock would quietly corrupt the attendance report and nobody would know why.
- Direction is computed server-side. The device does not know whether a person is inside; only the log knows. Keeping this on the server also means a firmware bug cannot invent a direction.
- Rules can be per person or per department, and exactly one of the two must be set — enforced by a CHECK constraint. A person-specific rule beats a department rule when both match.
- Authorisation lives in the server code, not the database. MySQL has no row-level security like some other systems, so decision.js is the single place the rules are applied.
- Card UIDs stored as uppercase hex text, not binary. Easier to read in Workbench and to compare with what the hardware team prints on the serial console. Slightly more storage, worth it.

7. How to recreate the database in MySQL Workbench

1. Connect to the local MySQL instance (127.0.0.1:3306).
2. Open db/01_schema.sql and run it. It drops and recreates attendance_system, so it is safe to run repeatedly.
3. Open db/02_seed.sql and run it for the sample data.
4. Check with: SELECT * FROM v_occupancy;

```
USE attendance_system;
SELECT * FROM v_occupancy;
SELECT * FROM access_events ORDER BY scanned_at DESC LIMIT 10;
```

8. The ER diagram

Generated from the live database rather than drawn by hand, so it cannot disagree with the schema: Database → Reverse Engineer → select attendance_system → finish. Arrange the boxes, then File → Export → Export as PNG and paste the image here.

[Paste the Workbench EER diagram screenshot here]

9. Questions I still have

- If someone forgets to tap out, should the system close the day for them automatically at a fixed time, or leave them showing as inside and flag it? I have gone with flagging for now, because silently closing it hides a real problem.
- Should a denied attempt at a restricted door alert anyone immediately, or is a report enough?
- How long do we keep events? An audit log that grows forever will eventually be slow. I would like to agree a retention period with the team.
- For visitors, is a day pass the right model, or should the pass expire a fixed number of hours after first use?